

Document number	P3811R0
Date	2025-08-15
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	SG23 Safety and Security

default comparison memory safety

Table of contents

- default comparison memory safety
 - Abstract
 - Motivation
 - Wording
 - Impact on the standard
 - References

Abstract

The C++ standard should require compilers to implement defaulted comparison operator functions free of memory safety related undefined behavior and this guarantee should be stated in the standard.

Is it unreasonable to require compilers to implement defaulted comparison operator functions free of memory safety related undefined behavior? Consider the sample generated examples from the proposals that created the defaulted comparison operator functions language feature.

Consistent comparison ^[1]

```
class Point {
    int x;
    int y;
public:
    friend bool operator==(const Point& a, const Point& b)
    {
        return a.x == b.x && a.y == b.y;
    }
    friend bool operator< (const Point& a, const Point& b)
    {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
    friend bool operator!=(const Point& a, const Point& b)
    {
        return !(a==b);
    }
    friend bool operator<=(const Point& a, const Point& b)
    {
        return !(b<a);
    }
    friend bool operator> (const Point& a, const Point& b)
    {
        return b<a;
    }
    friend bool operator>=(const Point& a, const Point& b)
    {
        return !(a<b);
    }
    // ... non-comparison functions ...
};
```

```

class TotallyOrdered : Base {
    string tax_id;
    string first_name;
    string last_name;
public:
    std::strong_ordering operator<=>(const TotallyOrdered& that) const {
        if (auto cmp = (Base&)(*this) <=> (Base&)that; cmp != 0) return cmp;
        if (auto cmp = last_name <=> that.last_name; cmp != 0) return cmp;
        if (auto cmp = first_name <=> that.first_name; cmp != 0) return cmp;
        return tax_id <=> that.tax_id;
    }
    // ... non-comparison functions ...
};

```

<=> != == [2]

```

struct X {
    A a;
    B b;
    C c;

    ??? operator<=>(X const& rhs) const {
        if (auto cmp = a <=> rhs.a; cmp != 0)
            return cmp;
        if (auto cmp = b <=> rhs.b; cmp != 0)
            return cmp;
        return c <=> rhs.c;
    }

    bool operator==(X const& rhs) const {
        return a == rhs.a &&
            b == rhs.b &&
            c == rhs.c;
    }

    bool operator!=(X const& rhs) const {
        return !(*this == rhs);
    }
};

```

When do you actually use <=>? [3]

```
struct Aggr {
    int i;
    char c;
    Legacy q;

    strong_ordering operator<=>(Aggr const& rhs) const {
        // pairwise <=> works fine for these
        if (auto cmp = i <=> rhs.i; cmp != 0) return cmp;
        if (auto cmp = c <=> rhs.c; cmp != 0) return cmp;

        // synthesizing strong_ordering from == and <
        if (q == rhs.q) return strong_ordering::equal;
        if (q < rhs.q) return strong_ordering::less;

        // sanitizers might also check for
        [[ assert: rhs.q < q; ]]
        return strong_ordering::greater;
    }
};
```

How do these examples rank with respect to being free of memory safety related undefined behavior?

memory safety	
☐☐☐☐	uninitialized memory
☐☐☐☐	range access
☐☐☐☐	null pointer dereference
☐☐☐☐	dangling
☐☐☐☐	invalidation
☐☐☐☐	type safety

Motivation

There has been sustained interest in creating islands of safety in C++ that goes beyond the constexpr evaluation of core constant expressions .

Safe C++ [4]

Strategy for removing safety-related UB by default [5]

Most every day programmers are more specifically concerned about the memory unsafety subset of undefined behavior. While we can't at this moment mandate that all of C++ is free of memory unsafety, can we at least lock down the low hanging fruit that is already amenable to safety related verbiage.

Wording

11.10 Comparisons [class.compare]

11.10.1 Defaulted comparison operator functions [class.compare.default]

¹ A defaulted comparison operator function (12.4.3) shall be a non-template function that

...

^{1.4} free of memory safety related undefined or erroneous behavior.

Impact on the standard

Safety not only requires safety to be in the language but also a part of the libraries even when those functions are created by the compiler. This proposal provides safety guarantees in some portions of the standard by requiring compiler implementers to make defaulted comparison operator functions free of memory safety related undefined behavior.

The proposed changes are relative to the current working draft N5008 [6].

References

1. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0515r3.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0515r3.pdf>) 
2. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1185r2.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1185r2.html>) 

3. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1186r3.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1186r3.html>) 
4. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3390r0.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3390r0.html>) 
5. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3436r0.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3436r0.pdf>) 
6. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>) 